PROYECTO FINAL — SEGUNDO PARCIAL

Sistema de Control de Inventario Inteligente

Dashboard de Ventas con Machine Learning y API REST

Harold Toribio
Programación con Inteligencia Artificial
2025

1. Descripción del Escenario

Una bodega enfrenta un problema crítico de organización de inventario: los costos de almacenamiento superan los niveles deseados y la minimización de costos no está siendo factible. Adicionalmente, el personal no cuenta con visibilidad sobre el manejo y control de productos cuando estos se encuentran escasos o faltantes, lo que genera pérdidas de ventas y clientes insatisfechos.

Este proyecto desarrolla un sistema inteligente de control de inventario que combina una base de datos MySQL, una API REST con Flask, modelos de Machine Learning y un dashboard interactivo para visualizar estadísticas y predicciones de demanda en tiempo real.

1.1 Objetivo General

Construir un sistema de extremo a extremo que permita gestionar el inventario de una bodega, predecir la demanda futura de productos usando modelos de regresión, y exponer esa información mediante una API REST consumida por un frontend interactivo.

1.2 Objetivos Específicos

- Diseñar una base de datos MySQL con stored procedures para todas las operaciones, eliminando el riesgo de inyección SQL.
- Implementar y comparar múltiples modelos de regresión (LinearRegression, DecisionTree, RandomForest, GradientBoosting) para predicción de demanda.
- Aplicar encoders de clasificación (LabelEncoder, OneHotEncoder, OrdinalEncoder) al feature engineering temporal.
- Exponer los modelos entrenados como endpoints REST usando Flask.
- Desarrollar un dashboard en HTML/JavaScript que consuma la API y muestre gráficas, estadísticas y predicciones.
- Generar historial simulado automáticamente para productos nuevos, usando Montecarlo en el stored procedure de MySQL.

2. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas claramente separadas, con una capa adicional de servicios ML:

Capa	Tecnología	Responsabilidad	Archivos clave
Frontend	HTML + JS + Chart.js	UI, gráficas, formularios	index.html, dashboard.js, style.css
API REST	Python + Flask	Endpoints HTTP, validación	api.py
Servicios ML	scikit-learn + pandas	Entrenamiento y predicción	modelo_ml.py, prediccion_service.py
Base de Datos	MySQL	Persistencia + stored procedures	DB_Control_Inventario.sql

2.1 Estructura de Carpetas

```
dashboard-ventas/
  ├── backend/
  │   ├── api.py                  ← Endpoints Flask
  │   ├── data_loader.py          ← Carga datos de MySQL a DataFrame
  │   ├── encoders.py             ← LabelEncoder, OneHot, Ordinal
  │   ├── database/
  │   │   └── connection.py       ← Conexión MySQL (env vars)
  │   ├── repositories/
  │   │   └── ventas_repository.py← Todas las llamadas a SP
  │   └── services/
  │       ├── modelo_ml.py        ← Entrenamiento y comparación de modelos
  │       └── prediccion_service.py
  ├── frontend/
  │   ├── index.html
  │   ├── dashboard.js
  │   └── style.css
  └── DB_Control_Inventario.sql
```

3. Diseño de la Base de Datos

3.1 Tablas

La base de datos InventarioSimulacion contiene dos tablas principales con una relación uno a muchos:

Tabla	Campo	Tipo	Descripción
productos	id_producto	INT PK AUTO	Identificador único
	nombre	VARCHAR(100)	Nombre del producto
	categoria	VARCHAR(50)	Categoría (nullable)
	precio	DECIMAL(10,2)	Precio unitario
ventas	id_venta	INT PK AUTO	Identificador único
	id_producto	INT FK	Referencia a productos
	fecha	DATE	Fecha de la venta
	cantidad	INT	Unidades vendidas

3.2 Stored Procedures

Todas las operaciones de base de datos se realizan exclusivamente a través de stored procedures. Esto elimina el riesgo de inyección SQL ya que ningún valor de usuario se concatena directamente en una query.

Stored Procedure	Parámetros	Descripción
ListarProductos()	Ninguno	Retorna todos los productos
CrearProducto()	nombre, precio	Inserta un nuevo producto
EliminarProducto()	id	Elimina producto y sus ventas
ObtenerVentasPorProducto()	id_producto	Ventas históricas de un producto
ObtenerResumenVentas()	Ninguno	Total por producto (JOIN)
ObtenerVentasTotales()	Ninguno	Todas las ventas con detalle
ObtenerDashboardResumen()	Ninguno	KPIs generales del dashboard
SimularVentasHistoricas()	id, promedio, dias	Genera historial simulado con variación aleatoria ±50%

3.3 Simulación de Historial para Productos Nuevos

Cuando el usuario agrega un producto desde el frontend, ingresa un promedio de ventas diarias estimado. El sistema llama automáticamente al stored procedure SimularVentasHistoricas, que genera 90 días de historial con cantidades aleatorias entre el 50% y el 150% del promedio indicado. Esto garantiza que el modelo ML tenga suficientes datos para realizar predicciones desde el primer momento.

4. Modelos de Machine Learning

4.1 Encoders de Clasificación

Antes de entrenar los modelos, las variables categóricas y temporales se transforman mediante tres encoders implementados en encoders.py:

- LabelEncoder: Transforma el día de la semana (texto) a valores numéricos del 0 al 6.
- OneHotEncoder: Codifica la columna 'mes' en variables binarias independientes para cada mes.
- OrdinalEncoder: Codifica el mes como valor ordinal numérico, conservando el orden temporal.

El feature engineering extrae las siguientes características de la columna fecha para alimentar los modelos:

Feature	Descripción	Rango de valores
dia	Día del mes	1 – 31
mes	Mes del año	1 – 12
dia_semana	Día de la semana	0 (lunes) – 6 (domingo)
trimestre	Trimestre del año	1 – 4

4.2 Modelos de Regresión Implementados

El sistema entrena y compara automáticamente cuatro modelos de regresión para predecir la demanda diaria de unidades. La variable objetivo (y) es la cantidad de unidades vendidas por día.

Modelo	Descripción	Ventaja principal
LinearRegression	Regresión lineal clásica	Interpretable, base de comparación
DecisionTreeRegressor	Árbol de decisión	Captura relaciones no lineales simples
RandomForestRegressor	Ensemble de árboles (100 estimadores)	Robusto ante overfitting
GradientBoostingRegressor	Boosting secuencial (100 estimadores)	Alta precisión en datos tabulares

4.3 Evaluación y Selección Automática

Los datos se dividen en 80% entrenamiento y 20% prueba. Cada modelo se evalúa con dos métricas:

- MAE (Mean Absolute Error): Error promedio absoluto entre predicción y valor real. Menor es mejor.

- R^2 (Coeficiente de determinación): Qué tan bien explica el modelo la varianza de los datos. Cercano a 1 es mejor.

El modelo con menor MAE se selecciona automáticamente para generar las predicciones futuras. Los resultados de todos los modelos se retornan al frontend y se muestran en una tabla comparativa.

4.4 Predicciones Futuras

Una vez seleccionado el mejor modelo, se generan predicciones para los próximos N días (7, 14 o 30 según elija el usuario). Las fechas futuras se calculan dinámicamente a partir de la fecha actual, por lo que las predicciones siempre son vigentes.

El endpoint retorna un JSON estructurado con: modelo seleccionado, comparación de todos los modelos, lista de predicciones con fecha y cantidad estimada, y estadísticas históricas del producto.

5. API REST con Flask

5.1 Endpoints disponibles

Método	Ruta	Parámetros	Descripción
GET	/productos	Ninguno	Lista todos los productos
POST	/productos	nombre, precio, promedio_ventas	Crea producto y genera historial simulado
DELETE	/productos/<id>	id en URL	Elimina producto y sus ventas
GET	/ventas	Ninguno	Todas las ventas con detalle
GET	/ventas/<id>	id en URL	Ventas de un producto específico
GET	/ventas-por-producto	Ninguno	Resumen agrupado por producto
GET	/prediccion/<id>	dias (query param, default 7)	Predicción ML + comparación de modelos
GET	/dashboard	producto, dias	KPIs generales del dashboard

5.2 Ejemplo de Respuesta del Endpoint de Predicción

```
GET /prediccion/1?dias=7
{
  "modelo_seleccionado": "RandomForest",
  "comparacion_modelos": {
    "LinearRegression":  { "MAE": 1.42, "R2": 0.81 },
    "DecisionTree":      { "MAE": 1.87, "R2": 0.74 },
    "RandomForest":      { "MAE": 1.21, "R2": 0.89 },
    "GradientBoosting":  { "MAE": 1.33, "R2": 0.86 }
  },
  "predicciones": [
    { "fecha": "2025-04-01", "prediccion": 8.5 },
    { "fecha": "2025-04-02", "prediccion": 7.3 }
  ],
  "estadisticas_historicas": {
    "promedio_diario": 8.2, "max_diario": 14,
    "min_diario": 3,   "total_vendido": 738, "dias_registrados": 90
  }
}
```

5.3 Seguridad: Variables de Entorno

Las credenciales de la base de datos no se almacenan en el código fuente. La clase DatabaseConnection lee las credenciales desde variables de entorno del sistema operativo:

- DB_HOST: servidor de MySQL (por defecto localhost)

- DB_USER: usuario de MySQL
- DB_PASSWORD: contraseña de MySQL
- DB_NAME: nombre de la base de datos

El repositorio incluye un archivo .env.example como plantilla y un .gitignore que excluye el archivo .env real del control de versiones.

6. Frontend — Dashboard Interactivo

El frontend es una aplicación de una sola página (SPA) construida con HTML, CSS y JavaScript vanilla, que consume todos los endpoints de la API mediante fetch() asincrónico. Las gráficas se renderizan con Chart.js.

6.1 Secciones del Dashboard

- **Dashboard: Tarjetas de KPIs (ventas totales, promedio, producto top, total de productos) y gráfica de torta con la distribución de ventas por producto.**
- **Productos: Formulario para agregar productos con nombre, precio y promedio de ventas estimado. Lista de productos existentes con opción de eliminar.**
- **Predicción ML: Selector de producto y días a predecir. Tabla comparativa de los 4 modelos con sus métricas MAE y R^2. Gráfica de línea con las predicciones futuras.**
- **Reportes: Gráfica de barras con ventas totales por producto.**

6.2 Flujo de la Sección de Predicción

1. El usuario selecciona un producto y la cantidad de días a predecir.
2. El frontend llama a GET /prediccion/<id>?dias=N.
3. Flask carga el historial de ventas desde MySQL vía stored procedure.
4. modelo_ml.py entrena los 4 modelos, evalúa con MAE/R^2 y selecciona el mejor.
5. El modelo seleccionado genera predicciones para los próximos N días.
6. El frontend muestra la tabla de modelos y la gráfica de predicciones.

7. Instrucciones de Ejecución

7.1 Requisitos

- Python 3.9 o superior
- MySQL 8.0 o superior
- Node.js (solo para regenerar el documento)
- Librerías Python: flask, flask-cors, mysql-connector-python, pandas, scikit-learn, numpy

7.2 Configuración de la Base de Datos

Ejecutar el archivo DB_Control_Inventario.sql en MySQL Workbench o desde la línea de comandos:

```
mysql -u root -p < DB_Control_Inventario.sql
```

Esto creará la base de datos, las tablas, los stored procedures y los datos de prueba.

7.3 Configuración del Backend

Crear las variables de entorno antes de correr Flask (en Windows usar set en lugar de export):

```
export DB_HOST=localhost
export DB_USER=root
export DB_PASSWORD=tu_password
export DB_NAME=inventariosimulacion
python -m flask --app backend.api run --debug
```

7.4 Ejecución del Frontend

Abrir el archivo frontend/index.html directamente en el navegador, o usar un servidor local:

```
cd frontend && python -m http.server 8080
```

Luego acceder a http://localhost:8080 en el navegador.

8. Conclusiones

El proyecto integra exitosamente los temas del curso en un sistema funcional de extremo a extremo:

- La transición de un dataset estático a una base de datos MySQL real con stored procedures demuestra buenas prácticas de acceso a datos y seguridad contra inyecciones SQL.
- La implementación de cuatro modelos de regresión con comparación automática de métricas permite seleccionar el modelo óptimo para cada conjunto de datos, en lugar de asumir un modelo predeterminado.
- El uso de encoders (LabelEncoder, OneHotEncoder, OrdinalEncoder) para el feature engineering temporal enriquece la representación de los datos y mejora la capacidad predictiva de los modelos.
- La arquitectura en capas (repositorio → servicio → API → frontend) facilita el mantenimiento y la extensión del sistema.
- La simulación de historial para productos nuevos mediante el stored procedure SimularVentasHistoricas resuelve el problema del “cold start”, permitiendo que el modelo ML pueda predecir desde el primer día de vida de un producto.

Repositorio: https://github.com/HaroldToribio/dashboard-ventas